How Netflix Built a Distributed Write Ahead Log For Its Data Platform

BYTEBYTEGO
DEC 02, 2025

Monster SCALE Summit 2026 (Sponsored)

Extreme Scale Engineering | Online | March 11-12

Your free ticket to Monster SCALE Summit is waiting — 30+ engineering talks on data-intensive applications

Monster SCALE Summit is a virtual conference that's all about extreme-scale engineering and data-intensive applications. Engineers from Discord, Disney,

LinkedIn, Pinterest, Rivian, American Express, Google, ScyllaDB, and more will be sharing 30+ talks on topics like:

- Distributed databases
- Streaming and real-time processing
- Intriguing system designs
- Massive scaling challenge

Don't miss this chance to connect with 20K of your peers designing, implementing, and optimizing data-intensive applications – for free, from anywhere.

Register now to save your seat, and become eligible for an early bird swag pack!

Disclaimer: The details in this post have been derived from the details shared online by the Netflix Engineering Team. All credit for the technical details goes to the Netflix Engineerin Team. The links to the original articles and sources are present in the references section at end of the post. We've attempted to analyze the details and provide our input about them. I you find any inaccuracies or omissions, please leave a comment, and we will do our best to them.

Netflix processes an enormous amount of data every second. Each time a user plays show, rates a movie, or receives a recommendation, multiple databases and microservices work together behind the scenes. This functionality is supported usi hundreds of independent systems that must stay consistent with each other. When something goes wrong in one system, it can quickly create a ripple effect across the platform.

Netflix's engineering team faced several recurring issues that threatened the reliabi of their data. Some of these included accidental data corruption after schema chang inconsistent updates between storage systems such as Apache Cassandra and

Elasticsearch, and message delivery failures during transient outages. At times, bul operations like large delete jobs even caused key-value database nodes to run out of memory. On top of that, some databases lacked built-in replication, which meant th regional failures could lead to permanent data loss.

Each engineering team tried to handle these issues differently. One team would bui custom retry systems, another would design its own backup strategy, and yet anoth would use Kafka directly for message delivery. While these solutions worked individually, they created complexity and inconsistent guarantees across Netflix's ecosystem. Over time, this patchwork approach increased maintenance costs and made debugging more difficult.

To fix this, Netflix built a Write-Ahead Log system to act as a single, resilient foundation for data reliability. The WAL standardizes how data changes are record stored, and replayed across services. In simple terms, it captures every change befor is applied to the database, so that even if something fails midway, no information is lost.

In this article, we will look at how Netflix built this WAL and the challenges it face

What is a Write-Ahead Log?

At its core, a Write-Ahead Log is a simple but powerful idea. It is a system that kee record of every change made to data before those changes are applied to the actual database. You can think of it like keeping a journal of all the actions you plan to tak Even if something goes wrong during the process, you still have that journal to rem you exactly what you were doing, so you can pick up right where you left off.

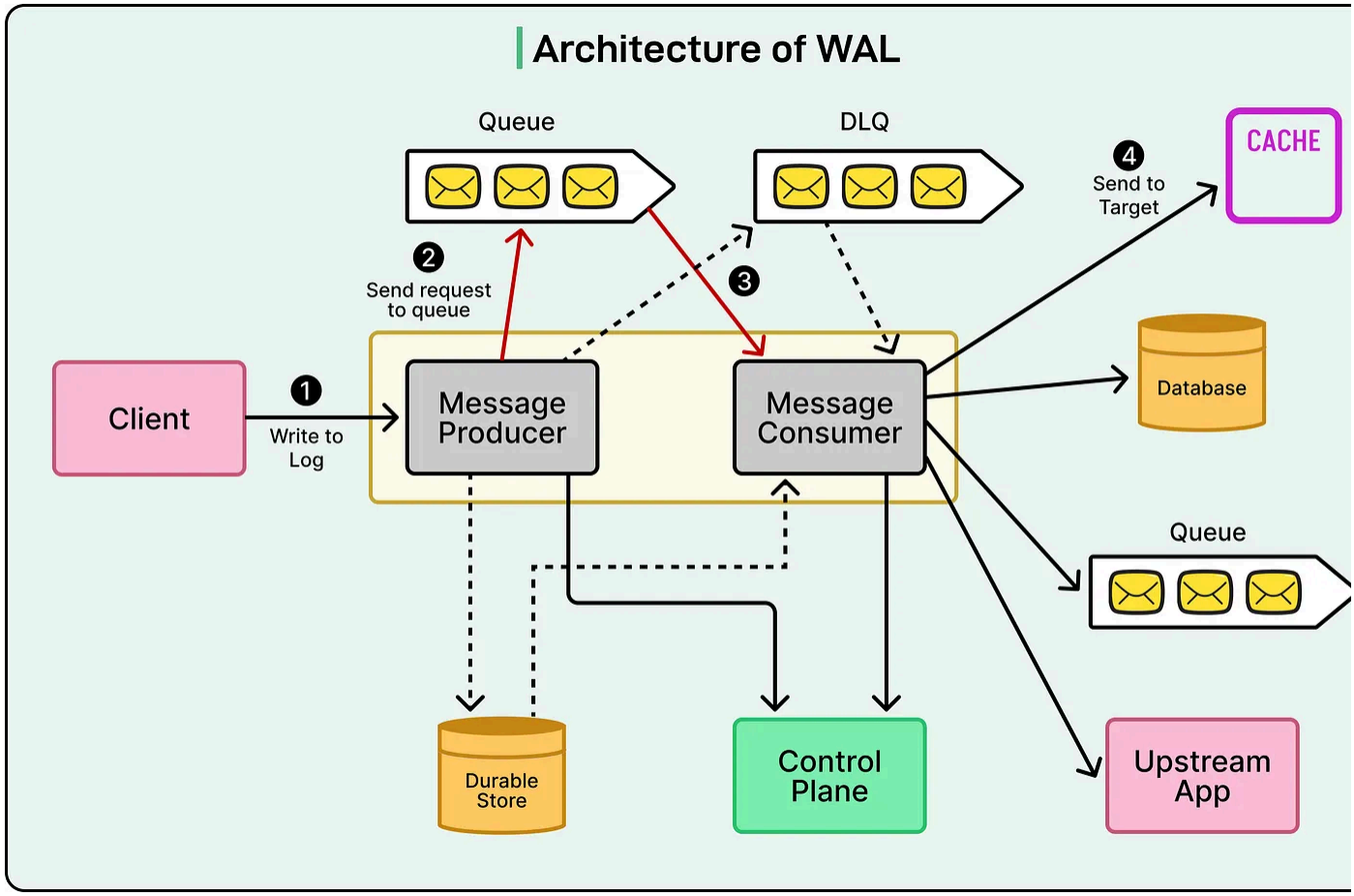
In practical terms, when an application wants to update or delete information in a database, it first writes that intention to the WAL. Only after the entry has been saf recorded does the database proceed with the operation. This means that if a server crashes or a network connection drops, Netflix can replay the operations from the

WAL and restore everything to the correct state. Nothing is lost, and the data remai consistent across systems.

Netflix's version of WAL is not tied to a single database or service.

It is distributed, meaning it runs across multiple servers to handle massive volumes data. It is also pluggable, allowing it to connect easily to various technologies, such Kafka, Amazon SQS, Apache Cassandra, and EVCache. This flexibility allowed the Netflix engineering team to use the same reliability framework for different types c workloads, whether it's storing cached video metadata, user preferences, or system logs.

See the diagram below:



The WAL provides several key benefits that make Netflix's data platform more resilient:

- **Durability:** Every change is logged first, so even if a database goes offline, no d is permanently lost.
- **Retry and Delay Support:** If a message fails to process due to an outage or network issue, the WAL can automatically retry it later, with custom delays.
- **Cross-Region Replication:** Data can be copied across regions, ensuring the san information exists in multiple data centers for disaster recovery.
- **Multi-Partition Consistency:** For complex updates involving multiple tables or partitions, WAL ensures that all changes are coordinated and eventually consistent.

The WAL API

Netflix's Write-Ahead Log system provides a simple interface for the developers. Despite the complexity of what happens behind the scenes, the API that developers interact with contains only one main operation called WriteToLog.

This API acts as the entry point for any application that wants to record a change. T structure looks something like this:

rpc WriteToLog (WriteToLogRequest) returns (WriteToLogResponse);

Even though this may look technical, the idea is straightforward. A service sends a request to WAL describing what it wants to write and where that data should go. W then processes the request, stores it safely, and responds with information about whether the operation was successful.

The request contains four main parts:

- **Namespace:** This identifies which logical group or application the data belong
  Think of it as a label that helps WAL organize and isolate data from different teams or services.
- **Lifecycle:** This specifies timing details, such as whether the message should be delayed or how long WAL should keep it.
- **Payload:** This is the actual content or data being written to the log.
- **Target:** This tells WAL where to send the data after it has been safely recorded such as a Kafka topic, a database, or a cache.

The response from WAL is equally simple:

- **Durable:** Indicates whether the request was successfully stored and made relial
- **Message:** Provides details if something went wrong, like an error message or reason for failure.

Each namespace in WAL has its own configuration that defines how it behaves. For example, one namespace may be set up to use Kafka for high-speed streaming, whil another might rely on Amazon SQS for delayed message delivery. The team can adj settings like retry counts, backoff times, and delay intervals depending on what eac application needs.

Different Use Cases of the WAL

Netflix designed the WAL system to be flexible enough to support many different situations, which they refer to as personas. Each persona represents a unique way t WAL is used within the company's data ecosystem.

Let's look at a few of the main ones to understand how this system adapts to differe needs.

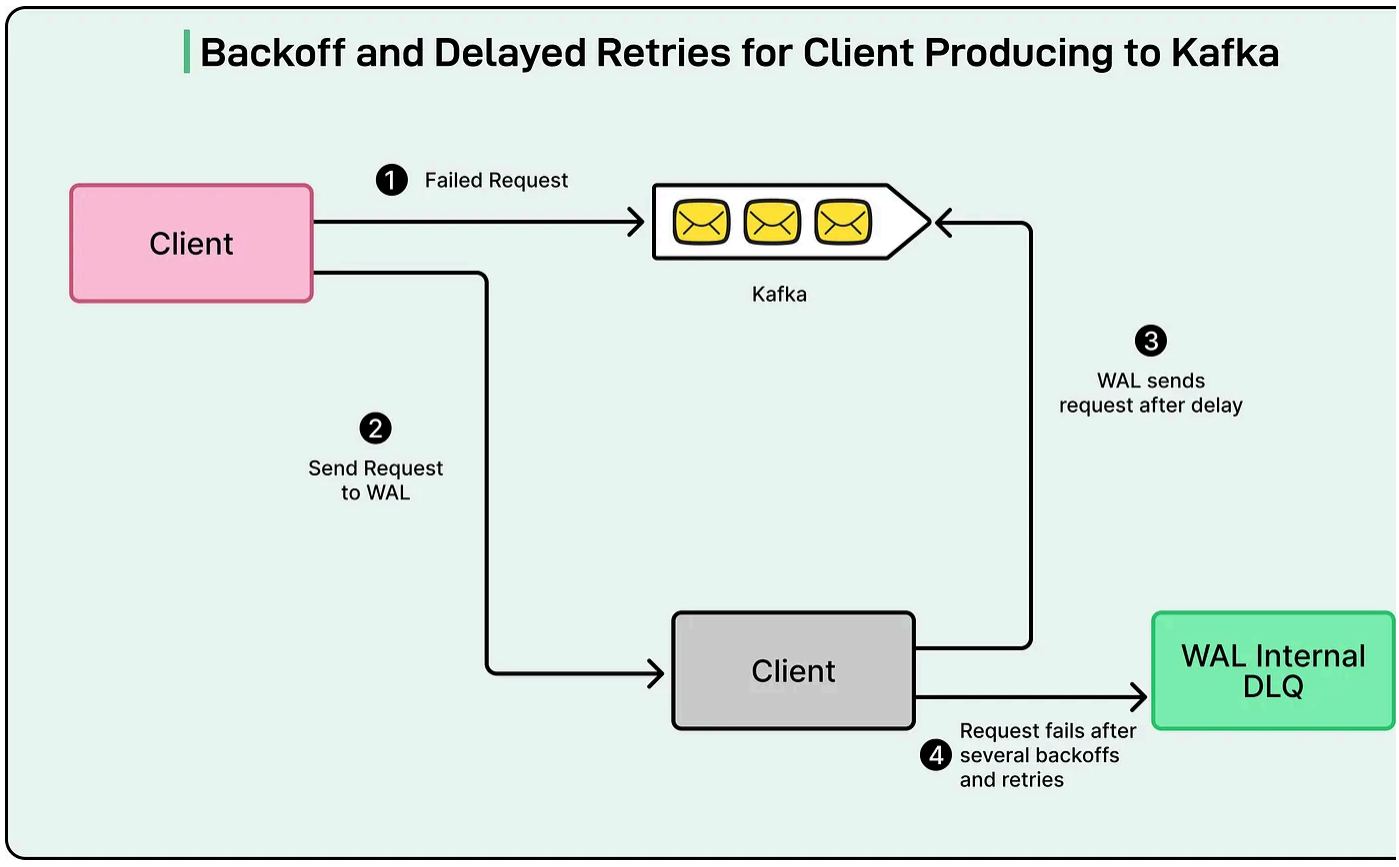
1 - Delayed Queues

This use case comes from the Product Data Systems (PDS) team, which handles a lc real-time data updates.

In large-scale systems like Netflix, failures are inevitable. Sometimes, a downstrean service such as Kafka or a database might be temporarily unavailable due to networ issues or maintenance.

Instead of losing messages or forcing engineers to manually retry failed operations, WAL automatically steps in. When a system failure occurs, WAL uses Amazon SQS (Simple Queue Service) to delay messages and retry them later.

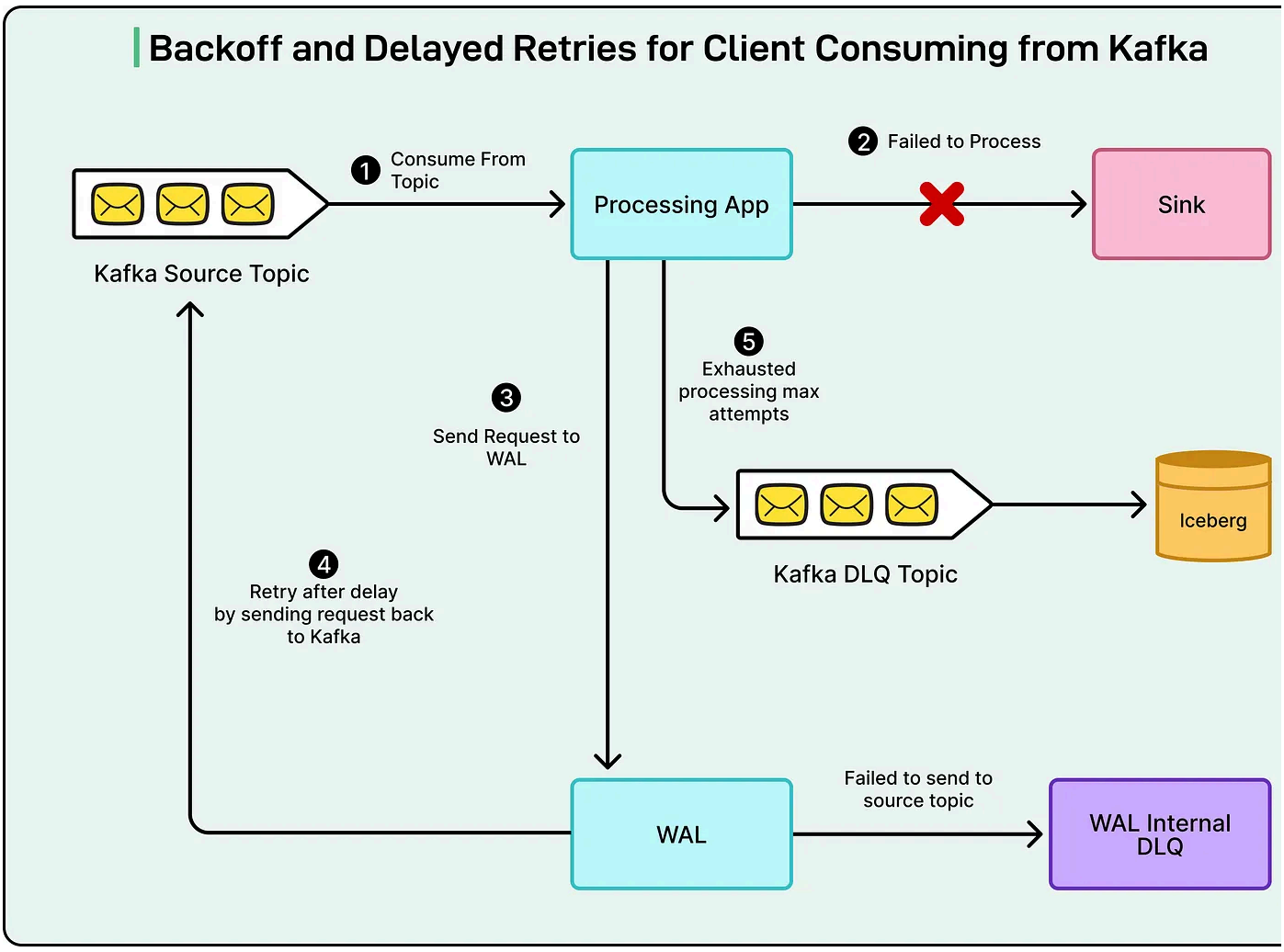
See the diagram below for backoff and delayed retries for clients producing to Kafk



Here's how it works in simple terms:

- If a message fails to be delivered, WAL stores it in a queue and waits for a certa amount of time before trying again. The delay can be configured based on how long the system is expected to recover.
- Once the downstream service is back online, WAL automatically retries the messages, ensuring nothing is lost and no manual intervention is needed.

The diagram below shows the backoff and delayed retries for clients consuming fro Kafka:



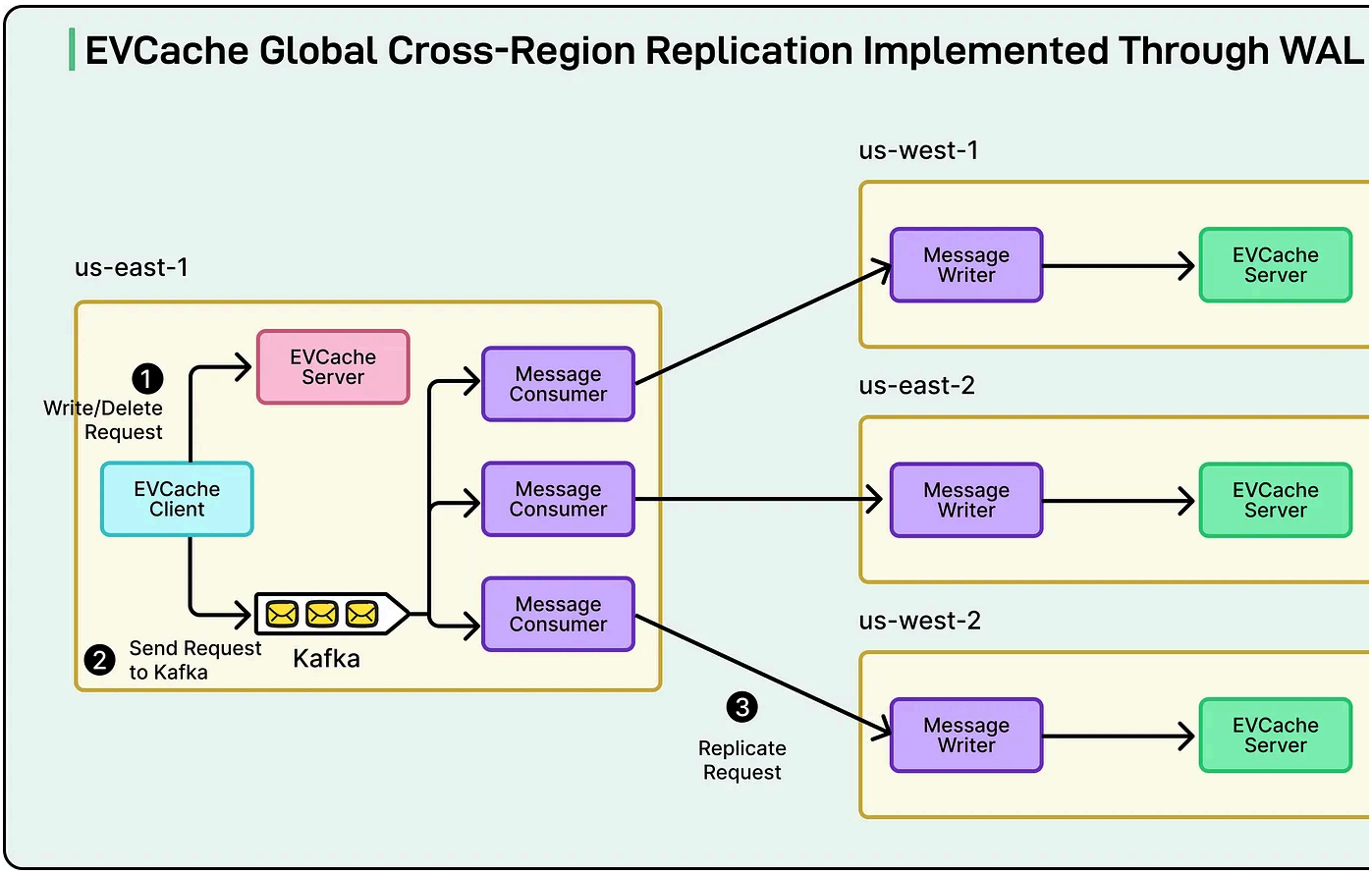
This approach saves engineers a lot of time and prevents cascading failures that mi otherwise spread across the platform.

2 - Cross-Region Replication

Another major use case is data replication across Netflix's global regions. The company's caching system, EVCache, stores frequently accessed data to make streaming fast and reliable. However, since Netflix operates worldwide, the same da needs to exist in multiple regions.

WAL makes this replication seamless by using Kafka under the hood. Whenever da is written or deleted in one region, WAL captures that event and sends it to other regions. The consumers in each region then replay the same operations locally, ensuring that all copies of the data stay synchronized.

See the diagram below:



In simpler terms, WAL acts like a reliable postman, making sure every region recei the same "letters" (data updates), even if network disruptions occur. This system ke

Netflix consistent around the world. Users in India, Europe, or the US all see the sa data at nearly the same time.

3 - Multi-Partition Mutations

The final example involves Netflix's Key-Value data service, which stores informati in systems like Apache Cassandra. Sometimes, a single operation might need to update data spread across multiple partitions or tables. Handling these multi-part changes is tricky, especially in distributed systems, because a failure in one partitio can leave others out of sync.

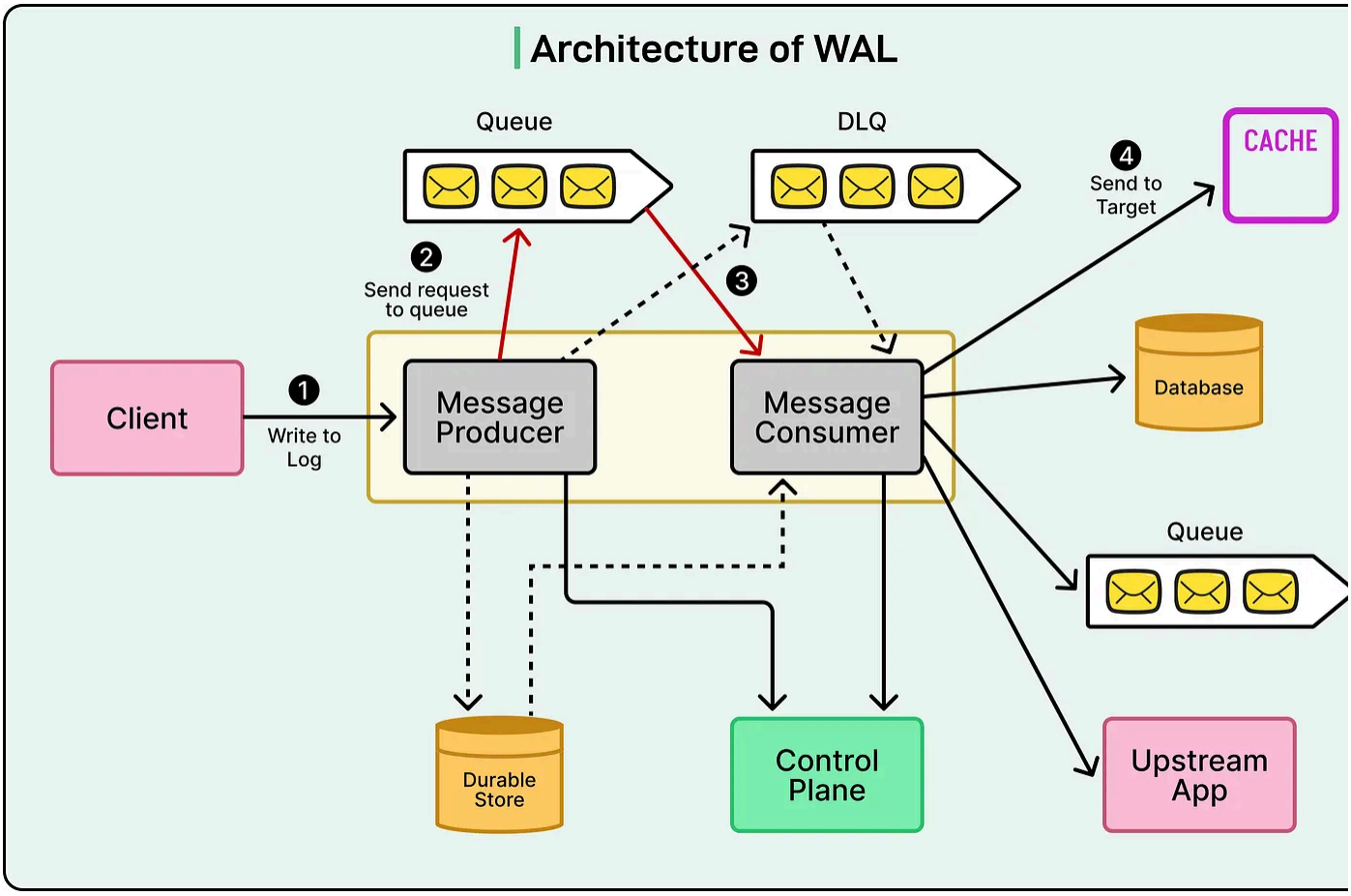
WAL solves this problem by ensuring atomicity, meaning that either all the change succeed or all are retried until they do. To achieve this, Netflix's WAL combines Ka for message delivery with durable storage for reliability. This setup functions simila to a two-phase commit, a well-known database technique that guarantees data consistency across multiple locations.

In short, WAL coordinates complex updates so that Netflix's data remains correct, even when multiple systems are involved.

Internal Architecture

To understand how Netflix's Write-Ahead Log (WAL) works behind the scenes, it helps to break it down into its main building blocks.

See the diagram below:



The system is made up of several key components that work together to move data safely from one place to another while keeping everything flexible and resilient.

- **Producers:** The producer is the first part of the system. It accepts messages or data change requests from various Netflix applications and writes them into a queue. You can think of producers as the “entry doors” of WAL. Whenever an a wants to log an update, it hands the data to a producer, which makes sure it get safely added to the right queue.
- **Consumers:** Consumers are the “exit doors” of the system. Their job is to read messages from the queue and send them to the correct destination, such as a database, cache, or another service. Since consumers run separately from producers, they can process messages at their own pace without slowing down rest of the system.

- **Message Queues:** The message queue is the middle layer that connects produce and consumers. Netflix primarily uses Kafka or Amazon SQS for this purpose. Each namespace in WAL (which represents a specific use case or service) has it own dedicated queue. This ensures isolation between applications so that a hea workload from one service does not affect another. Every namespace also inclu a Dead Letter Queue (DLQ). The DLQ is a special backup queue that stores messages that repeatedly fail to process. This gives engineers a chance to inspe and fix the problematic data later without losing it.
- **Control Plane:** The control plane is like the central command center for WAL. allows Netflix engineers to change settings, such as which queue type to use, h many retries should occur, or what the delay between retries should be. The ke advantage here is that teams can modify these settings without having to chan their application code. This makes the system highly adaptable and easy to maintain.
- **Targets:** Finally, the targets are the destinations where WAL sends the data. A target can be a database like Cassandra, a cache like EVCache, or even another message queue. The flexibility of defining targets through configuration means that the same WAL architecture can support many different workloads across Netflix.

Deployment Model

The way Netflix deploys its Write-Ahead Log (WAL) system is just as important as how it works internally.

To handle billions of data operations across many teams and services, Netflix neede platform that could scale easily, stay secure, and run reliably across regions. To achi this, WAL is deployed on top of Netflix's Data Gateway Infrastructure.

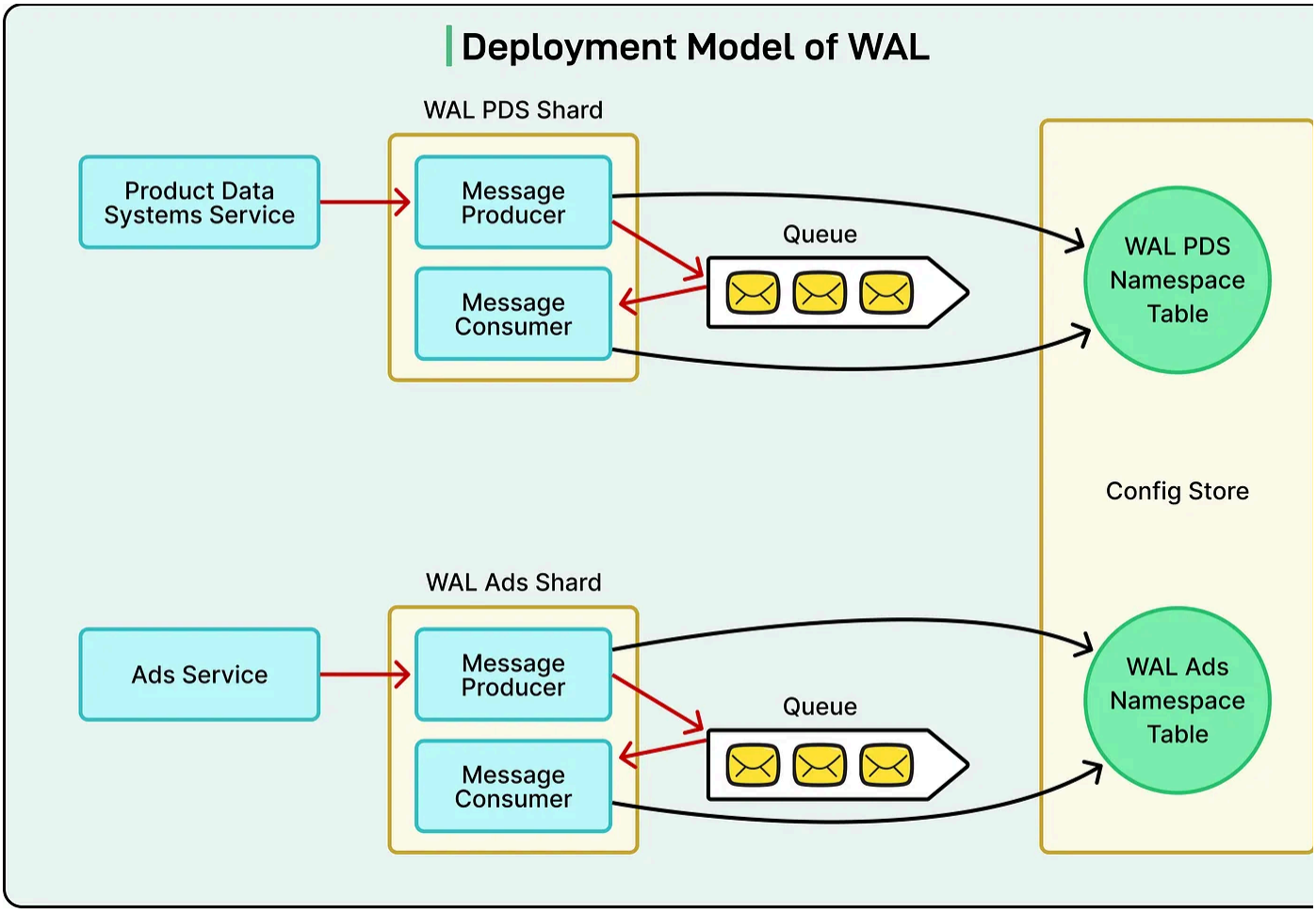
This infrastructure acts as a foundation that gives WAL several built-in advantages right out of the box:

- **mTLS for security:** All communication between services is encrypted and authenticated using mutual Transport Layer Security (mTLS). This ensures tha only trusted Netflix services can talk to each other, keeping sensitive data safe.
- **Connection management:** The platform automatically manages network connections, making sure requests are routed efficiently and that no single component gets overloaded.
- **Auto-scaling and load shedding:** WAL uses adaptive scaling to adjust the num of active instances based on demand. If CPU or network usage gets too high, th system automatically adds more capacity. In extreme cases, it can also shed low priority requests to protect the stability of the service.

Netflix organizes WAL deployments into shards. A shard is an independent deployment that serves a specific group of applications or use cases. For example, o shard might handle the Ads service, another might handle Gaming data, and so on. This separation prevents the “noisy neighbor” problem, where one busy service cou slow down others running on the same system.

Inside each shard, there can be multiple namespaces, each with its own configurati and purpose. These configurations are stored in a globally replicated SQL database ensuring they are always available and consistent, even if a region goes offline.

See the diagram below for the deployment model of WAL at Netflix:



Conclusion

Several key design principles shaped the success of WAL. The first is its pluggable architecture, which allows Netflix to switch between different technologies, such a Kafka or Amazon SQS, without changing application code. This flexibility ensures t teams can choose the most suitable underlying system for their specific use cases while relying on the same core framework.

Another principle is the reuse of existing infrastructure. Instead of building everyth from scratch, Netflix built WAL on top of its already established systems, like the D Gateway platform and Key-Value abstractions. This approach saved development ti and allowed the new system to fit naturally into the company's broader data architecture.

Equally important is the separation of concerns between producers and consumers. Because these components scale independently, Netflix can adjust each one based c traffic patterns or system load. This independence allows WAL to handle massive spikes in demand without service degradation.

Finally, Netflix recognizes that even a system designed for reliability must consider own limits. The team continuously evaluates trade-offs, such as dealing with slow consumers or managing backpressure during heavy traffic. Techniques like partitioning and controlled retries are essential to keeping the system stable.

Looking ahead, Netflix plans to enhance WAL further. Future improvements includ adding secondary indices to the Key-Value service, which will make data retrieval faster and more efficient, and supporting multi-target writes, allowing a single operation to send data to multiple destinations, such as a database and a backup system at the same time.

References:

- Building a Resilient Data Platform with Write-Ahead Log at Netflix
- Write-Ahead Logging

Ad spots typically sell out about 4 weeks in advance. To ensure your ad reaches this influential audience, reserve your space now by emailing sponsorship@bytebytego.com.

222 Likes · 15 Restacks

Discussion about this post

Comments Restacks

Write a comment...